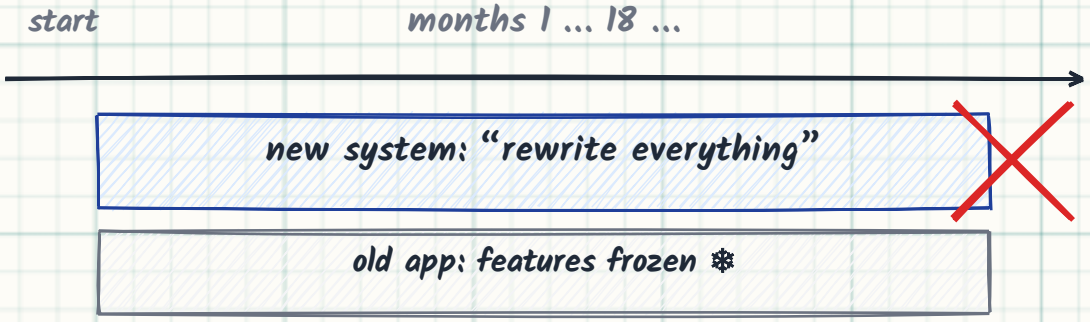
Breaking a Monolith

Lesson 4 · service boundaries & the Strangler Fig pattern

In this lesson → ① Why not big-bang ② Capability vs layer ③ Good boundaries ④ What to extract first ⑤ Strangler Fig ⑥ Moving the data ⑦ Safety techniques ⑧ ShopNest plan ⑨ Interview Q&A

Running project: ShopNest, our e-commerce app (users, products, orders, payments, notifications).

① Why not a big-bang rewrite?

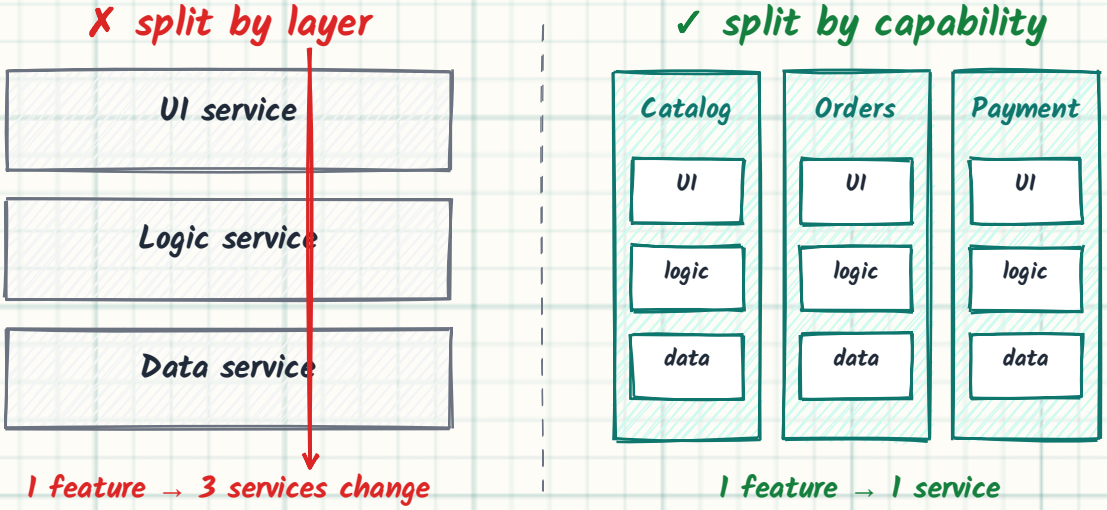


never finished · business waits · risk explodes

- ✗ Features frozen for months → business waits
- ✗ Two systems to maintain, requirements keep moving
- ✗ Huge risk at go-live, often **never finished**

> **Rule**
> Migrate **incrementally**: small steps, each one released and running in production.

② Split by capability, not by layer

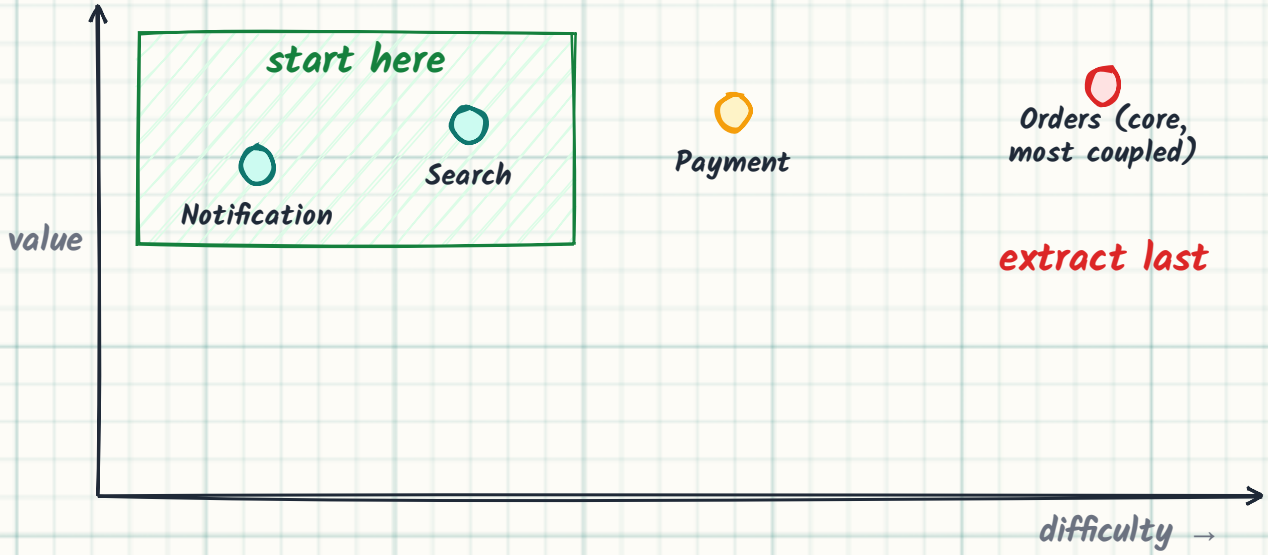


> **Business capability** = something the business does to create value: manage catalog, take orders, collect payments, ship parcels. Each capability owns its UI, logic and data.

③ 5 checks for a good boundary

Check	Question to ask	ShopNest example
High cohesion	Do these things change together?	Cart + checkout rules → Ordering
Low coupling	How many calls to other services?	Notification only listens to events
Data ownership	Who writes these tables?	Only Payment writes payments
Bounded context	Is the language consistent inside?	"Product" in Catalog vs Shipping
Team ownership	Can one team own it end to end?	Payments team owns Payment service

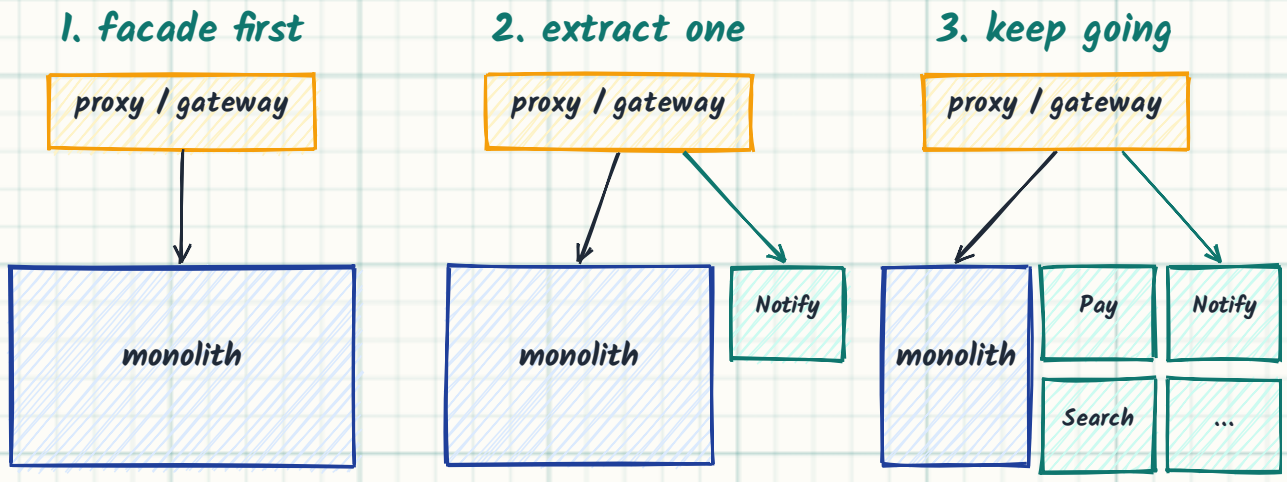
④ What to extract first?



→ Start at the **edge**: few dependencies, low risk (Notification)
→ Next: clear boundary or own scaling need (Search, Payment)
→ Last: the **core**, most connected part (Orders)

⑤ The Strangler Fig pattern

> **Strangler Fig** (Martin Fowler, 2004) = replace a legacy system piece by piece: new services grow around the old app until it can be switched off. Named after fig vines that grow around a host tree.



the monolith shrinks until it can be switched off

1. Put a **facade** (proxy / API gateway) in front of the monolith
2. Build **one** new service for one capability
3. **Route** that capability's traffic to the new service
4. **Delete** the old code from the monolith
5. **Repeat** until the monolith can be switched off

Why it works
The system runs at every step, and you can **roll back one route** if something breaks.

6 Moving the data



1) extract code · 2) move tables · 3) cut over

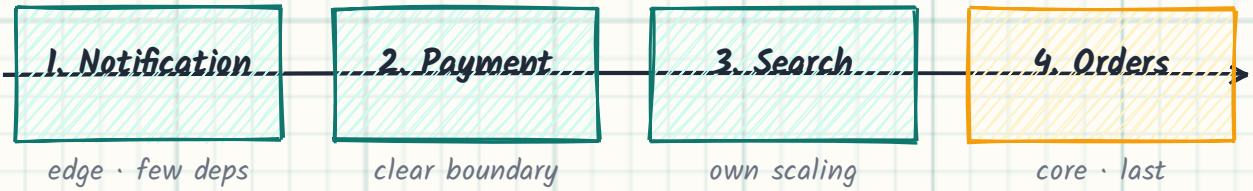
1. **Extract the code** first (it may read monolith data via an API for a short time)
2. **Move the tables** into the service's own database
3. **Sync** both sides during the move: domain events or **CDC** (change data capture)
4. **Cut over**, then drop the old tables

> **Careful**
> Avoid writing to two databases in one request (dual writes): one write can fail. Safe patterns like the **transactional outbox** come in Lesson 13.

7 3 safety techniques

Technique	How it works
Branch by abstraction	Put an interface in front of old code, build the new implementation behind it, then switch
Feature toggles	Enable the new route for a few users; switch off instantly if it breaks
Parallel run	Send the same request to old and new code; compare results before trusting the new one

⑧ ShopNest migration plan



Order	Service	Why this order
1	Notification	Edge service, only listens to events, low risk → build pipeline & monitoring
2	Payment	Clear boundary, external gateway already separate
3	Search	Needs its own scaling and its own store
4	Orders	Core and most connected: extract last, with experience

⑨ Anti-patterns

- ✗ **Big-bang rewrite**
- ✗ **Nano-services**: too many tiny services too soon
- ✗ **Shared database forever**: services split, data not
- ✗ Splitting by **technical layer**

Remember!
Extract only when a service brings **real value**: independent release, scaling or team ownership.

⑩ Interview Corner

Q. How would you migrate a monolith to microservices?

A. Incrementally with **Strangler Fig**: facade in front, extract one capability, route its traffic, delete old code, repeat.

Q. How do you choose service boundaries?

A. By business capability / bounded context, with high cohesion, low coupling and clear data ownership.

Q. Which service would you extract first, and why?

A. An edge service with few dependencies and low risk (e.g. Notification), to build the platform and confidence.

Q. Why avoid a big-bang rewrite?

A. Features freeze for months, risk is huge, and it often never finishes.

Q. What is branch by abstraction?

A. Introduce an interface over old code, build the new implementation behind it, then switch over.

⚡ Revise in 60 seconds

- → Split by **capability**, not layer
- → Cohesion · coupling · **own data**
- → Start at the **edge**, core last
- → Data: code → tables → sync → cut over
- → **Strangler Fig**: Fowler 2004
- → Facade → extract → route → delete → repeat
- → Branch by abstraction · toggles · parallel run
- → Avoid the **big-bang rewrite**

Next lesson → MS-05
Synchronous communication: **REST vs gRPC**, and why long call chains are dangerous.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**